Q1 - Q6 Answers

Name: Dac Vinh Nguyen

D: 101113852

Question 1: ReLU Neuron [10 marks]

Given values from the diagram:

Variable	Value
x_1	0.5
x_2	0.7
w_1	3
w_2	-2.0
b	0.35

The neuron computes a weighted sum (pre-activation) followed by an activation function:

$$z = \sum_i w_i x_i + b = w_1 x_1 + w_2 x_2 + b$$

The ReLU activation function is defined as:

$$\text{ReLU}(z) = \max(0,\, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Forward pass:

$$z = w_1 x_1 + w_2 x_2 + b = (0.5)(3) + (0.7)(-2.0) + 0.35 = 1.5 - 1.4 + 0.35 = 0.45$$

$$y = \text{ReLU}(z) = \text{ReLU}(0.45) = \max(0,\, 0.45) = \mathbf{0.45}$$

(a) $\partial y / \partial x_2$ and $\partial y / \partial w_2$ [3 marks]

The derivative of ReLU is:

$$\frac{\partial\, \text{ReLU}(z)}{\partial z} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Since $z = 0.45 > 0$, the neuron is in the **active region**, so:

$$\frac{\partial y}{\partial z} = 1$$

Applying the **chain rule** $\frac{\partial y}{\partial \theta} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial \theta}$, and noting that $\frac{\partial z}{\partial w_i} = x_i$ and $\frac{\partial z}{\partial x_i} = w_i$:

$$\boxed{\frac{\partial y}{\partial w_2} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_2} = 1 \cdot x_2 = \mathbf{0.7}}$$

$$\boxed{\frac{\partial y}{\partial x_2} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial x_2} = 1 \cdot w_2 = \mathbf{-2.0}}$$

(b) Effect of doubling w_1 on $\partial y / \partial x_1$ [1 mark]

Currently:

$$\frac{\partial y}{\partial x_1} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial x_1} = 1 \cdot w_1 = 1 \cdot 3 = 3$$

If w_1 doubles to $w_1' = 6$, we first verify the neuron remains active:

$$z' = w_1' x_1 + w_2 x_2 + b = (6)(0.5) + (-2.0)(0.7) + 0.35 = 3.0 - 1.4 + 0.35 = 1.95 > 0$$

Since $z' > 0$, ReLU is still active and $\frac{\partial y}{\partial z} = 1$. Therefore:

$$\left.\frac{\partial y}{\partial x_1}\right|_{w_1'=6} = 1 \cdot w_1' = 1 \cdot 6 = 6$$

$$\boxed{\text{The gradient doubles from 3 to 6.}}$$

This is because $\frac{\partial y}{\partial x_1} = w_1$ (when the neuron is active), so the gradient w.r.t. an input is **directly proportional** to its corresponding weight.

(c) Sigmoid activation [6 marks]

Replace ReLU with the **sigmoid** function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The pre-activation $z = 0.45$ remains unchanged. The forward pass gives:

$$y = \sigma(0.45) = \frac{1}{1 + e^{-0.45}} = \frac{1}{1 + 0.6376} = \frac{1}{1.6376} \approx \mathbf{0.6106}$$

The derivative of the sigmoid function is:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

$$\sigma'(0.45) = 0.6106 \times (1 - 0.6106) = 0.6106 \times 0.3894 \approx 0.2378$$

Now applying the chain rule, with $\frac{\partial y}{\partial z} = \sigma'(z)$:

$$\boxed{\frac{\partial y}{\partial w_1} = \sigma'(z) \cdot \frac{\partial z}{\partial w_1} = \sigma'(z) \cdot x_1 = 0.2378 \times 0.5 = \mathbf{0.1189}}$$

$$\boxed{\frac{\partial y}{\partial w_2} = \sigma'(z) \cdot \frac{\partial z}{\partial w_2} = \sigma'(z) \cdot x_2 = 0.2378 \times 0.7 = \mathbf{0.1665}}$$

Comparison with ReLU gradients:

Gradient	ReLU	Sigmoid
$\partial y/\partial w_1$	$x_1 = 0.5$	0.1189
$\partial y/\partial w_2$	$x_2 = 0.7$	0.1665

The sigmoid gradients are significantly smaller because $\sigma'(z) = 0.2378 < 1$ always holds (since $\max \sigma'(z) = 0.25$ at $z = 0$). This **saturation effect** causes gradients to diminish, making sigmoid harder to train in deep networks compared to ReLU.

Question 2: Neural Network Backpropagation [10 marks]

From the diagram, the network architecture is:

- **Inputs:** $x_1 = 1.2$, $x_2 = -0.6$
- **Hidden layer 1:** nodes a_1, a_2 (ReLU activation)
- **Hidden layer 2:** nodes a_3, a_4 (ReLU activation)
- **Output:** y (ReLU activation)

Note: to match the hint $y = 13.4$, I infer the weights $a_1 \to a_3$ and $a_1 \to a_4$ to be 2.

Weight table (inferred from diagram):

Connection	Weight	Connection	Weight
w_1: $x_1 \to a_1$	1	w_5: $a_1 \to a_3$	2
w_2: $x_2 \to a_1$	-1	w_6: $a_2 \to a_3$	2
w_3: $x_1 \to a_2$	-1	w_7: $a_1 \to a_4$	2
w_4: $x_2 \to a_2$	4	w_8: $a_2 \to a_4$	-1
b_{a_1}	0	w_9: $a_3 \to y$	3
b_{a_2}	0.5	w_{10}: $a_4 \to y$	1
b_{a_3}	-1	b_y	1
b_{a_4}	1		

Forward Pass

For each neuron, we compute $z = \sum_i w_i \cdot \text{input}_i + b$, then apply $\text{ReLU}(z) = \max(0, z)$.

Layer 1:

$$z_{a_1} = w_1 x_1 + w_2 x_2 + b_{a_1} = (1)(1.2) + (-1)(-0.6) + 0 = 1.2 + 0.6 = 1.8$$

$$a_1 = \text{ReLU}(1.8) = \mathbf{1.8}$$

$$z_{a_2} = w_3 x_1 + w_4 x_2 + b_{a_2} = (-1)(1.2) + (4)(-0.6) + 0.5 = -1.2 - 2.4 + 0.5 = -3.1$$

$$a_2 = \text{ReLU}(-3.1) = \mathbf{0} \quad (\text{dead neuron: } z_{a_2} < 0)$$

Layer 2:

$$z_{a_3} = w_5 \cdot a_1 + w_6 \cdot a_2 + b_{a_3} = (2)(1.8) + (2)(0) + (-1) = 3.6 + 0 - 1 = 2.6$$

$$a_3 = \text{ReLU}(2.6) = \mathbf{2.6}$$

$$z_{a_4} = w_7 \cdot a_1 + w_8 \cdot a_2 + b_{a_4} = (2)(1.8) + (-1)(0) + 1 = 3.6 + 0 + 1 = 4.6$$

$$a_4 = \text{ReLU}(4.6) = \mathbf{4.6}$$

Output layer:

$$z_y = w_9 \cdot a_3 + w_{10} \cdot a_4 + b_y = (3)(2.6) + (1)(4.6) + 1 = 7.8 + 4.6 + 1 = 13.4$$

$$\boxed{y = \text{ReLU}(13.4) = \mathbf{13.4}} \quad \checkmark \text{ (matches hint)}$$

Backward Pass

We use backpropagation to compute $\frac{\partial y}{\partial w_i}$ for every weight. Define the **local error signal** (delta) for each neuron j:

$$\delta_j = \frac{\partial y}{\partial z_j} = \frac{\partial y}{\partial a_j} \cdot \text{ReLU}'(z_j)$$

where $\text{ReLU}'(z_j) = 1[z_j > 0]$ (1 if active, 0 if dead).

The gradient of any weight is then:

$$\frac{\partial y}{\partial w_{i \to j}} = \delta_j \cdot \text{input}_i$$

Step 1 — Output layer:

Starting from the output, $\frac{\partial y}{\partial y} = 1$ and $z_y = 13.4 > 0$, so:

$$\delta_y = 1 \cdot \text{ReLU}'(13.4) = 1 \cdot 1 = 1$$

Output layer weight gradients:

$$\boxed{\frac{\partial y}{\partial w_9} = \delta_y \cdot a_3 = 1 \times 2.6 = \mathbf{2.6}} \quad \checkmark \text{ (matches hint)}$$

$$\frac{\partial y}{\partial w_{10}} = \delta_y \cdot a_4 = 1 \times 4.6 = \mathbf{4.6}$$

$$\frac{\partial y}{\partial b_y} = \delta_y = \mathbf{1}$$

Step 2 — Propagate deltas to layer 2:

Each hidden node receives error from all downstream nodes it connects to:

$$\frac{\partial y}{\partial a_3} = \delta_y \cdot w_9 = 1 \times 3 = 3$$

$$\frac{\partial y}{\partial a_4} = \delta_y \cdot w_{10} = 1 \times 1 = 1$$

Applying the ReLU derivative (both $z_{a_3} = 2.6 > 0$ and $z_{a_4} = 4.6 > 0$, so both active):

$$\delta_{a_3} = \frac{\partial y}{\partial a_3} \cdot \mathrm{ReLU}'(z_{a_3}) = 3 \times 1 = 3$$

$$\delta_{a_4} = \frac{\partial y}{\partial a_4} \cdot \mathrm{ReLU}'(z_{a_4}) = 1 \times 1 = 1$$

Layer 2 weight gradients:

$$\frac{\partial y}{\partial w_5} = \delta_{a_3} \cdot a_1 = 3 \times 1.8 = \mathbf{5.4}$$

$$\frac{\partial y}{\partial w_6} = \delta_{a_3} \cdot a_2 = 3 \times 0 = \mathbf{0}$$

$$\frac{\partial y}{\partial b_{a_3}} = \delta_{a_3} = \mathbf{3}$$

$$\frac{\partial y}{\partial w_7} = \delta_{a_4} \cdot a_1 = 1 \times 1.8 = \mathbf{1.8}$$

$$\frac{\partial y}{\partial w_8} = \delta_{a_4} \cdot a_2 = 1 \times 0 = \mathbf{0}$$

$$\frac{\partial y}{\partial b_{a_4}} = \delta_{a_4} = \mathbf{1}$$

Step 3 — Propagate deltas to layer 1:

Each hidden node in layer 1 receives error from all nodes in layer 2 it connects to (summing over outgoing paths):

$$\frac{\partial y}{\partial a_1} = \delta_{a_3} \cdot w_5 + \delta_{a_4} \cdot w_7 = 3 \times 2 + 1 \times 2 = 6 + 2 = 8$$

$$\frac{\partial y}{\partial a_2} = \delta_{a_3} \cdot w_6 + \delta_{a_4} \cdot w_8 = 3 \times 2 + 1 \times (-1) = 6 - 1 = 5$$

Applying the ReLU derivative:

- a_1 is **active** ($z_{a_1} = 1.8 > 0$): $\delta_{a_1} = 8 \times 1 = 8$
- a_2 is **dead** ($z_{a_2} = -3.1 < 0$): $\delta_{a_2} = 5 \times 0 = 0$

Layer 1 weight gradients:

$$\boxed{\frac{\partial y}{\partial w_1} = \delta_{a_1} \cdot x_1 = 8 \times 1.2 = \mathbf{9.6}} \quad \checkmark \text{ (matches hint)}$$

$$\frac{\partial y}{\partial w_2} = \delta_{a_1} \cdot x_2 = 8 \times (-0.6) = \mathbf{-4.8}$$

$$\frac{\partial y}{\partial b_{a_1}} = \delta_{a_1} = \mathbf{8}$$

Since $\delta_{a_2} = 0$, **all gradients into a_2 are zero** (the dead neuron blocks all gradient flow):

$$\frac{\partial y}{\partial w_3} = \delta_{a_2} \cdot x_1 = 0, \quad \frac{\partial y}{\partial w_4} = \delta_{a_2} \cdot x_2 = 0, \quad \frac{\partial y}{\partial b_{a_2}} = 0$$

Step 4 — Input gradients:

$$\boxed{\frac{\partial y}{\partial x_1} = \delta_{a_1} \cdot w_1 + \delta_{a_2} \cdot w_3 = 8 \times 1 + 0 \times (-1) = \mathbf{8}} \quad \checkmark \text{ (matches hint)}$$

$$\frac{\partial y}{\partial x_2} = \delta_{a_1} \cdot w_2 + \delta_{a_2} \cdot w_4 = 8 \times (-1) + 0 \times 4 = \mathbf{-8}$$

Summary of all gradients

Weight	Gradient	Weight	Gradient
$\partial y / \partial w_1$	9.6 ✓	$\partial y / \partial w_5$	5.4
$\partial y / \partial w_2$	-4.8	$\partial y / \partial w_6$	0
$\partial y / \partial w_3$	0	$\partial y / \partial w_7$	1.8
$\partial y / \partial w_4$	0	$\partial y / \partial w_8$	0
$\partial y / \partial w_9$	2.6 ✓	$\partial y / \partial w_{10}$	4.6
$\partial y / \partial x_1$	8 ✓	$\partial y / \partial x_2$	-8

Key observation: Node a_2 is a **dead ReLU** neuron ($z_{a_2} < 0$). Because $\mathrm{ReLU}'(z) = 0$ for $z < 0$, the error signal $\delta_{a_2} = 0$, meaning no gradient flows through a_2 — none of its incoming weights (w_3, w_4, b_{a_2}) receive any update signal. This illustrates the "dying ReLU" problem.

Question 3: RMSProp and ADAM [5 marks]

RMSProp

Standard **Stochastic Gradient Descent (SGD)** uses a fixed learning rate for all parameters:

$$w_{t+1} = w_t - \eta\, g_t \quad \text{where } g_t = \nabla_w L(w_t)$$

AdaGrad improves on this by adapting the learning rate per-parameter, accumulating all past squared gradients:

$$G_t = \sum_{\tau=1}^{t} g_\tau^2, \qquad w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}}\, g_t$$

However, G_t grows monotonically, so the effective learning rate $\frac{\eta}{\sqrt{G_t}}$ eventually shrinks to near zero and training stalls.

RMSProp (Root Mean Square Propagation) solves this by replacing the cumulative sum with an **exponentially decaying moving average** of squared gradients:

$$v_t = \beta\, v_{t-1} + (1 - \beta)\, g_t^2$$

$$\boxed{w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}}\, g_t}$$

where $\beta \in [0, 1)$ is the decay rate (typically $\beta = 0.9$) and $\epsilon \approx 10^{-8}$ prevents division by zero.

The denominator $\sqrt{v_t}$ approximates the RMS (root mean square) of recent gradients. Parameters with consistently large gradients get a smaller effective learning rate $\frac{\eta}{\sqrt{v_t}}$, and parameters with small gradients get a larger one — this **adaptive per-parameter scaling** stabilises training across dimensions with different gradient magnitudes.

ADAM (Adaptive Moment Estimation)

ADAM combines the adaptive learning rate of RMSProp with a **momentum-like first moment estimate**. It tracks two exponential moving averages:

First moment (mean of gradients — analogous to momentum):

$$m_t = \beta_1\, m_{t-1} + (1 - \beta_1)\, g_t$$

Second moment (mean of squared gradients — analogous to RMSProp):

$$v_t = \beta_2\, v_{t-1} + (1 - \beta_2)\, g_t^2$$

Since both $m_0 = 0$ and $v_0 = 0$, these estimates are **biased towards zero** in early steps ($\mathbb{E}[m_t] = (1 - \beta_1^t)\,\mathbb{E}[g_t]$). ADAM corrects this with:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The parameter update is then:

$$\boxed{w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\,\hat{m}_t}$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.001$, $\epsilon = 10^{-8}$.

Key Differences: RMSProp vs ADAM

Aspect	RMSProp	ADAM
Update direction	Raw gradient g_t	Smoothed first moment $\hat{m}_t$
Adaptive scaling	$v_t = \beta v_{t-1} + (1 - \beta) g_t^2$	$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
Momentum	No	Yes (via m_t)
Bias correction	No	Yes (via $\hat{m}_t$, $\hat{v}_t$)
Update formula	$-\frac{\eta}{\sqrt{v_t + \epsilon}} g_t$	$-\frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$

ADAM replaces the raw gradient g_t with the smoothed first moment $\hat{m}_t$, so the update direction considers the **recent history of gradients** (like momentum), making convergence smoother. The bias correction ensures accurate moment estimates in early training, which RMSProp lacks.

Question 4: ReLU and Tanh Neurons [10 marks]

ReLU Neuron [5 marks]

A neuron computes a pre-activation as a weighted sum of its inputs:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

A **ReLU (Rectified Linear Unit)** neuron then applies the activation:

$$f(z) = \mathrm{ReLU}(z) = \max(0,\, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Output range: $f(z) \in [0, +\infty)$

Derivative (gradient of the output w.r.t. the pre-activation):

$$\frac{\partial f}{\partial z} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$

At $z = 0$, the derivative is technically undefined (sub-gradient); in practice it is set to 0.

The gradient of the loss L w.r.t. any weight w_i via the chain rule is:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial w_i} = \begin{cases} \frac{\partial L}{\partial f} \cdot x_i & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Advantages:

- **No vanishing gradient** for $z > 0$: the derivative is exactly 1, so gradients pass through unchanged. This makes training deep networks much easier compared to sigmoid/tanh.
- **Computationally efficient:** only requires a threshold operation.
- **Sparsity:** neurons with $z < 0$ output exactly 0, leading to sparse activations.

Disadvantage — Dying ReLU problem: If $z < 0$, the gradient is 0, so the neuron receives no weight updates. If a large negative bias or unfortunate weight initialization causes $z < 0$ for all training inputs, the neuron becomes permanently dead. Variants like Leaky ReLU ($f(z) = \max(\alpha z, z)$ with small $\alpha > 0$) address this.

Hyperbolic Tangent (tanh) Neuron [5 marks]

A **tanh** neuron applies:

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

This can also be expressed in terms of the sigmoid function $\sigma(z)$:

$$\tanh(z) = 2\sigma(2z) - 1$$

Output range: $f(z) \in (-1, +1)$, centred at zero.

Derivative:

$$\frac{\partial f}{\partial z} = 1 - \tanh^2(z) = 1 - f(z)^2$$

Key properties of the derivative:

- At $z = 0$: $\tanh(0) = 0$, so $f'(0) = 1 - 0^2 = 1$ (maximum gradient)
- As $|z| \to \infty$: $\tanh(z) \to \pm 1$, so $f'(z) \to 1 - 1 = 0$ (**saturation**)

The gradient of the loss w.r.t. a weight:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial w_i} = \frac{\partial L}{\partial f} \cdot \left(1 - f(z)^2\right) \cdot x_i$$

Advantages:

- **Zero-centred** output: unlike sigmoid ($\in (0, 1)$), tanh outputs are centred around 0. This means gradient updates to weights are not constrained to be all positive or all negative, leading to more efficient optimisation.
- Stronger gradients than sigmoid: $\max f'(z) = 1$ for tanh vs. $\max f'(z) = 0.25$ for sigmoid.

Disadvantage — Vanishing gradient: When $|z|$ is large, the gradient $1 - \tanh^2(z) \approx 0$, so deep networks suffer from vanishing gradients during backpropagation. In a network with L layers, the gradient at layer 1 involves a product $\prod_{l=1}^{L} f'(z^{(l)})$, which shrinks exponentially if each $f'(z^{(l)}) < 1$.

Comparison: ReLU vs Tanh

Property	ReLU	Tanh
Definition	$\max(0, z)$	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$
Output range	$[0, +\infty)$	$(-1, +1)$
Zero-centred	No	Yes
Derivative ($z > 0$)	1 (constant)	$1 - \tanh^2(z) \in (0, 1)$
Saturation	None for $z > 0$	Yes, for large $\|z\|$
Vanishing gradient	No (when active)	Yes (at saturation)
Dead neurons	Yes (dying ReLU)	No
Computation	$O(1)$ threshold	Exponential function

Question 5: Regularisation [5 marks]

Regularisation techniques prevent **overfitting** — when a model fits the training data too closely and fails to generalise to unseen data. The general objective with regularisation is:

$$\min_{w} \; L_{\text{total}}(w) = L_{\text{data}}(w) + \Omega(w)$$

where L_{data} is the data loss and $\Omega(w)$ is a regularisation penalty.

Dropout

During each training step, dropout independently sets each neuron's activation to zero with probability p (the dropout rate). Formally, for a layer with activations $\mathbf{h}$:

$$r_i \sim \text{Bernoulli}(1 - p) \quad \text{for each neuron } i$$

$$\tilde{h}_i = r_i \cdot h_i$$

where $r_i \in \{0, 1\}$ is a random mask. When $r_i = 0$, neuron i is "dropped" — its output and all downstream gradient flow are zeroed out.

Why it works: Each training step effectively trains a **different sub-network** (with n neurons, there are 2^n possible sub-networks). The final model is implicitly an **ensemble average** of all these sub-networks, which reduces variance and overfitting.

At test time, no neurons are dropped ($r_i = 1$ for all i), but we must compensate for the fact that more neurons are now active. The expected activation during training is:

$$\mathbb{E}[\tilde{h}_i] = (1 - p) \cdot h_i$$

So at test time, activations are scaled by $(1 - p)$:

$$h_i^{\text{test}} = (1-p) \cdot h_i$$

Equivalently, during training one can use **inverted dropout**, scaling activations up by $\frac{1}{1-p}$ so that no scaling is needed at test time:

$$\tilde{h}_i^{\text{train}} = \frac{r_i}{1-p} \cdot h_i$$

L2 Regularisation (Weight Decay)

L2 regularisation adds a penalty proportional to the **squared L2 norm** of the weight vector:

$$L_{\text{total}} = L_{\text{data}} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 = L_{\text{data}} + \frac{\lambda}{2} \sum_i w_i^2$$

where $\lambda > 0$ is the regularisation strength.

Why it works: The penalty $\frac{\lambda}{2}\|\mathbf{w}\|_2^2$ encourages weights to stay small, favouring smoother, less complex models that are less prone to overfitting. Geometrically, it constrains the weight vector to lie within a ball of bounded radius.

Gradient Adjustment with $\lambda = 0.001$

Taking the gradient of the total loss with respect to a single weight w_i:

$$\frac{\partial L_{\text{total}}}{\partial w_i} = \frac{\partial L_{\text{data}}}{\partial w_i} + \frac{\partial}{\partial w_i}\left(\frac{\lambda}{2} w_i^2\right) = \frac{\partial L_{\text{data}}}{\partial w_i} + \lambda\, w_i$$

With $\lambda = 0.001$, the gradient becomes:

$$\frac{\partial L_{\text{total}}}{\partial w_i} = \frac{\partial L_{\text{data}}}{\partial w_i} + 0.001 \cdot w_i$$

The SGD update rule then gives:

$$\boxed{w_i \leftarrow w_i - \eta\left(\frac{\partial L_{\text{data}}}{\partial w_i} + 0.001 \cdot w_i\right) = (1 - 0.001\eta)\, w_i - \eta \frac{\partial L_{\text{data}}}{\partial w_i}}$$

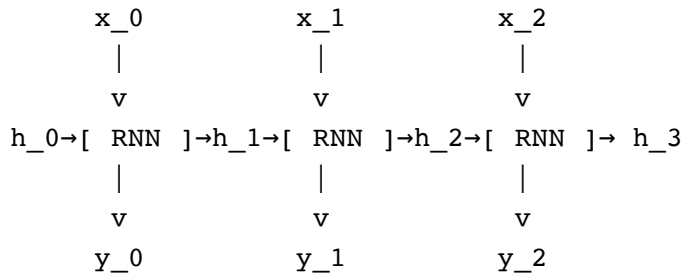
The term $(1 - 0.001\eta)$ multiplies the weight at each step, shrinking it towards zero — this is why L2 regularisation is called **"weight decay"**. The factor $0.001 \cdot w_i$ adds an extra gradient component pointing back towards the origin, penalising large weights proportionally to their magnitude.

Question 6: RNN and LSTM [10 marks]

Recurrent Neural Network (RNN)

An RNN processes sequences by maintaining a **hidden state** $h_t \in \mathbb{R}^d$ that is updated at each time step t. The hidden state acts as the network's "memory" of what it has seen so far.

Diagram — unrolled RNN over 3 time steps:



Each RNN cell computes: h_t = tanh(W_xh · x_t + W_hh · h_{t-1} + b_h)

Equations: At each time step t, the RNN cell computes:

$$h_t = \tanh\big(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h\big)$$

$$y_t = W_{hy}\, h_t + b_y$$

where:

- $x_t \in \mathbb{R}^m$ is the input at time t
- $h_t \in \mathbb{R}^d$ is the hidden state at time t
- $W_{xh} \in \mathbb{R}^{d\times m}$, $W_{hh} \in \mathbb{R}^{d\times d}$, $W_{hy} \in \mathbb{R}^{k\times d}$ are shared weight matrices
- The **same weights** (W_{xh}, W_{hh}, W_{hy}) are shared across all time steps — this is the "recurrent" structure.

Vanishing/Exploding gradient problem: During Backpropagation Through Time (BPTT), the gradient of the loss at time T w.r.t. the hidden state at time t involves a product of Jacobians:

$$\frac{\partial h_T}{\partial h_t} = \prod_{\tau=t+1}^{T} \frac{\partial h_\tau}{\partial h_{\tau-1}} = \prod_{\tau=t+1}^{T} \operatorname{diag}\big(1 - h_\tau^2\big)\, W_{hh}$$

For long sequences ($T - t$ large), this product tends to either:

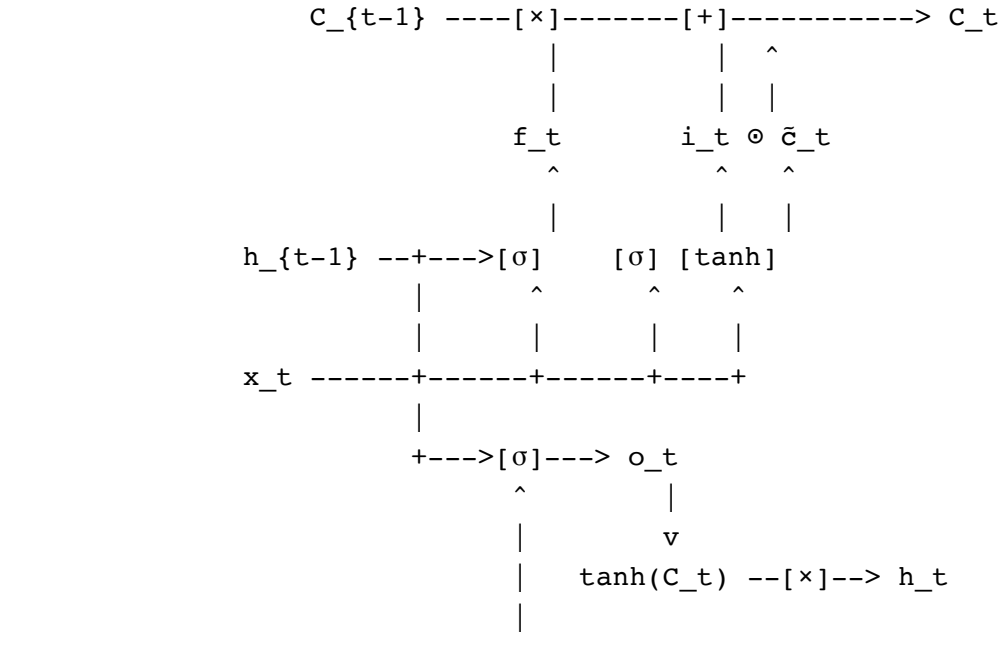
- **Vanish** ($\to 0$) if $\|W_{hh}\| < 1$ or activations are saturated ($h_\tau \approx \pm 1$)
- **Explode** ($\to \infty$) if $\|W_{hh}\| > 1$

This makes it practically impossible for standard RNNs to learn **long-range dependencies**.

Long Short-Term Memory (LSTM)

LSTM solves the vanishing gradient problem by introducing a **cell state** C_t — a separate memory path that carries information across many time steps via **additive** updates rather than multiplicative.

Diagram — LSTM cell:



The LSTM cell has three **gates**, each controlled by a sigmoid $\sigma(\cdot) \in (0, 1)$:

1. Forget gate f_t — decides what to **erase** from the cell state:

$$f_t = \sigma\big(W_f \cdot [h_{t-1},\, x_t] + b_f\big) \in (0,1)^d$$

2. Input gate i_t + **candidate** $\tilde{C}_t$ — decides what new information to **write**:

$$i_t = \sigma\big(W_i \cdot [h_{t-1},\, x_t] + b_i\big) \in (0,1)^d$$

$$\tilde{C}_t = \tanh\big(W_C \cdot [h_{t-1},\, x_t] + b_C\big) \in (-1,1)^d$$

Cell state update (the core equation):

$$\boxed{C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t}$$

Here $f_t \odot C_{t-1}$ selectively **forgets** old information, and $i_t \odot \tilde{C}_t$ selectively **writes** new information (where $\odot$ denotes element-wise/Hadamard product).

3. Output gate o_t — decides what to **output** from the cell state:

$$o_t = \sigma\big(W_o \cdot [h_{t-1},\, x_t] + b_o\big) \in (0,1)^d$$

$$h_t = o_t \odot \tanh(C_t)$$

Why Sigmoid for the Gates?

The gates **must** use the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ because:

1. **Output range** $(0, 1)$**:** Sigmoid outputs act as **soft switches/valves**:
   - $\sigma(\cdot) \approx 0$: gate is **closed** — information is blocked

- $\sigma(\cdot) \approx 1$: gate is **open** — information passes through

2. **Element-wise control:** Each dimension of the gate vector independently controls the corresponding dimension of the cell state, allowing fine-grained selective memory.
3. **Differentiability:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, so the gates are smoothly differentiable and can be learned via backpropagation.

For example, if $f_t^{(j)} \approx 0$ for dimension j, then $C_t^{(j)} \approx i_t^{(j)} \tilde{C}_t^{(j)}$ — the old memory in dimension j is erased. If $f_t^{(j)} \approx 1$ and $i_t^{(j)} \approx 0$, then $C_t^{(j)} \approx C_{t-1}^{(j)}$ — the memory is **perfectly preserved** across the time step.

Why LSTM Solves the Vanishing Gradient Problem

Consider the gradient of the cell state across time steps:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t + \frac{\partial (i_t \odot \tilde{C}_t)}{\partial C_{t-1}}$$

The key term is f_t: when the forget gate is open ($f_t \approx 1$), the gradient flows through **undiminished**. Over $T - t$ time steps:

$$\frac{\partial C_T}{\partial C_t} \approx \prod_{\tau=t+1}^{T} f_\tau$$

If the network learns to keep $f_\tau \approx 1$ (i.e., "remember"), the gradient product stays close to 1 instead of vanishing. This **additive** cell state update $C_t = f_t \odot C_{t-1} + \ldots$ contrasts with the RNN's **multiplicative** hidden state update $h_t = \tanh(W_{hh} h_{t-1} + \ldots)$, where the repeated multiplication by W_{hh} causes gradients to vanish or explode.

RNN vs LSTM — Summary

Property	RNN	LSTM
Hidden state update	$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
Memory mechanism	Single hidden state h_t	Separate cell state C_t + hidden state h_t
Update type	Multiplicative (via W_{hh})	Additive (via $+$ in cell state)
Gradient flow	$\prod \text{diag}(1 - h_\tau^2) W_{hh}$ — vanishes/explodes	$\prod f_\tau$ — can stay ≈ 1
Long-range dependencies	Poor	Good
Parameters	W_{xh}, W_{hh}, W_{hy}	W_f, W_i, W_C, W_o (4× more)
Gating	None	3 gates (f_t, i_t, o_t)